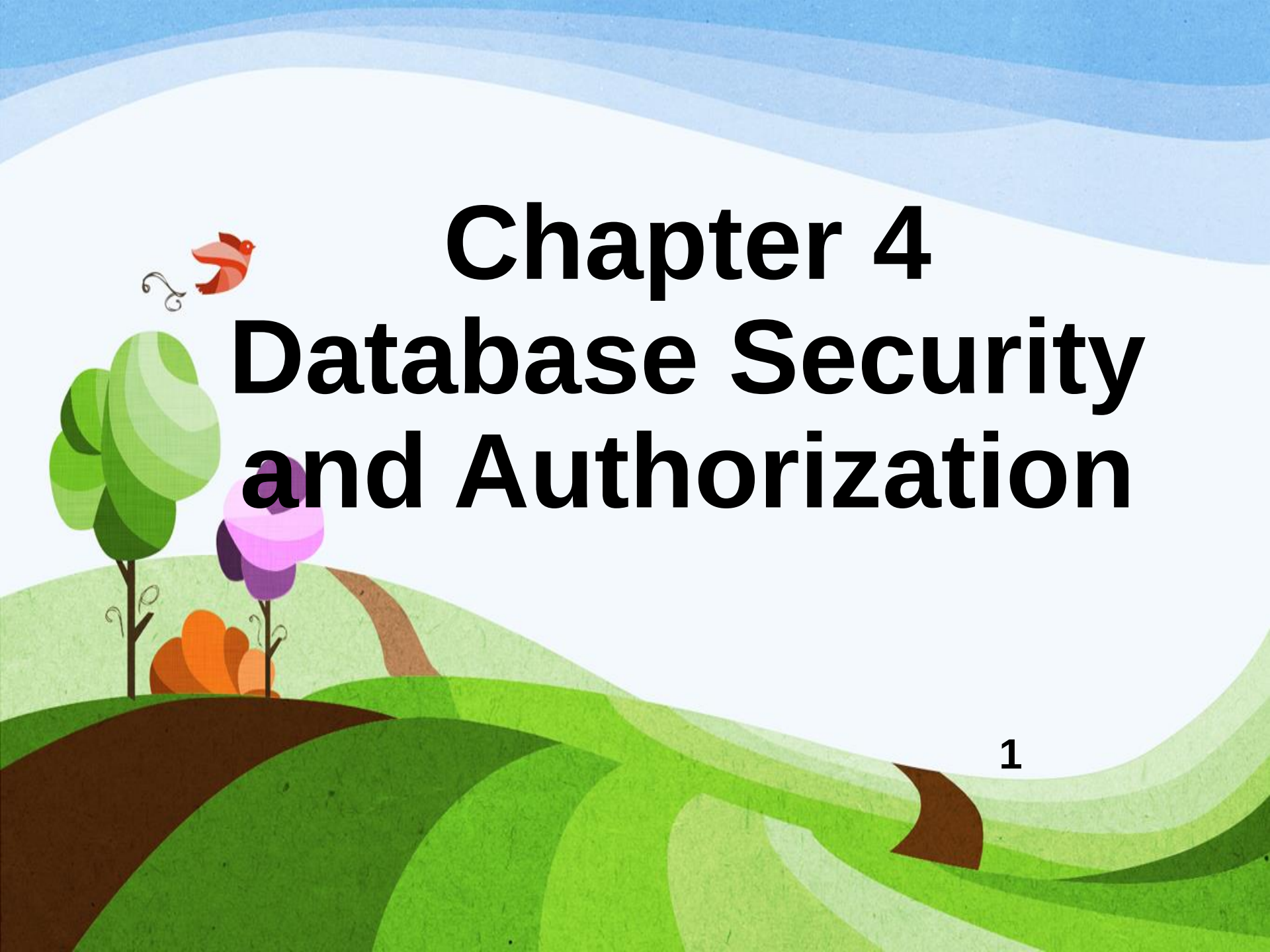


Chapter 4
Database Security and Authorization

Chapter Outline

- Introduction to Database Security
- Authentication
- Authorization
- Principals (Logins, Database Users, Roles)
- Managing Permissions
  - ✓Securables
  - ✓Permissions
- Data Encryption
- Stored Procedures and Triggers

Introduction to Database Security

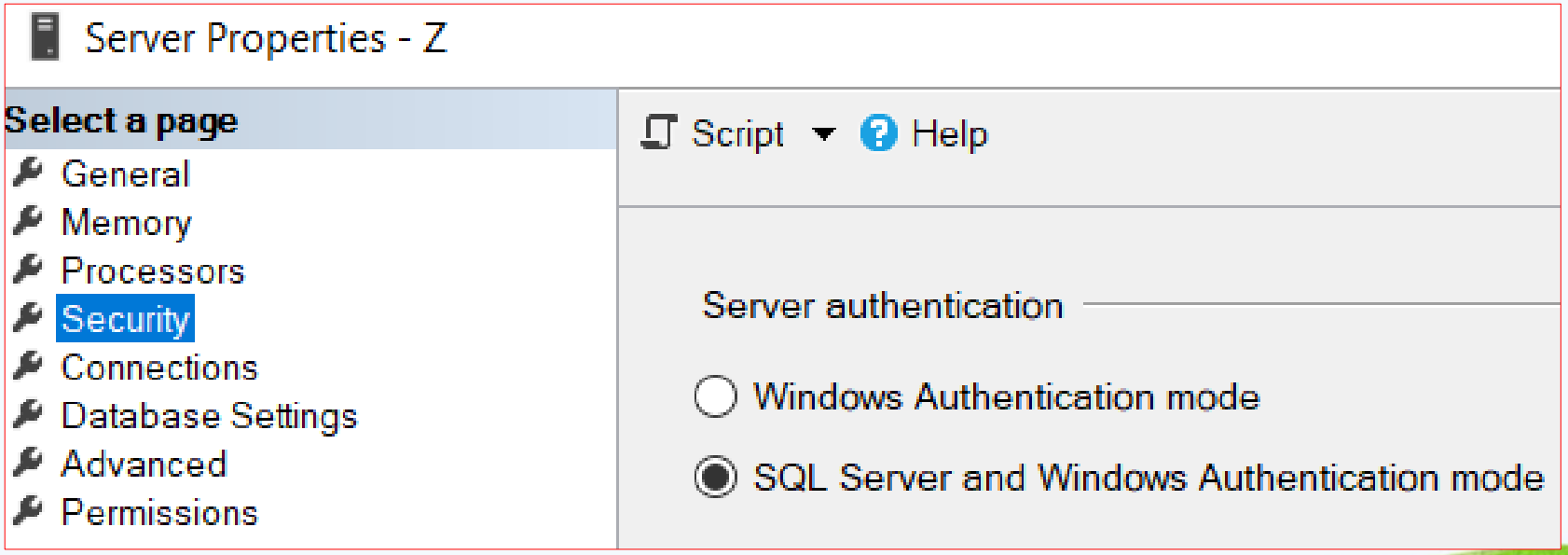
- Database security is the protection of database data against accidental and intentional loss or unauthorized access to data.
- Some common database security mechanisms:
  - Authentication
  - Authorization to manage database access:
    - ✓ GRANT, DENY and REVOKE commands
    - ✓ View
  - Encryption

Authentication

- **Authentication** is a mechanism that determines whether a user is who he or she claims to be.
- Authentication is used to obtain a positive identification of the user using:
  - User accounts and passwords
  - Strong authentication:
    - ✓ Smart card plus PIN
    - ✓ Biometric devices – use of fingerprints, voice, hair style, retinal scans, etc.

SQL Server Authentication Mode

- There are two types of authentication mode:
  1. Windows Authentication mode
  2. Mixed mode authentication(SQL Server and Windows Authentication mode)



SQL Server Authentication Mode…

- If you configure the authentication to Windows authentication mode, users can login(connect) to server using windows accounts only.
- If mixed mode authentication mode is configured, users can login(connect) to server using either windows accounts or SQL Server login accounts.
- When you change the authentication mode, you have to **restart** the **SQL Server instance** before the mode change takes effect.

Authorization

- **Authorization** is granting of a right or privilege that enables users and programs to have legitimate access to a system's object or resources.
- Restricts access to data and actions that people can take on data.
- Common forms of authorization on parts of the database include: Select, Insert, Update and Delete.

Creating Principals

Principals

- A security principal is a person or an object that needs access to a database.
- Principals are the means by which you authenticate and are identified within an instance or database.
- Principals are broken down into the following major categories:
  - Logins
  - Users
  - Roles

Principals (Cont'd…)

- The scope of influence of a principal depends on the scope of the definition of the principal:
  1) **Windows-level principals:** have a Windows-level permission scope.
  2) **Server-level principals:** have server-level permissions.
  3) **Database-level principals:** have database-level permissions.

Principals (Cont'd…)

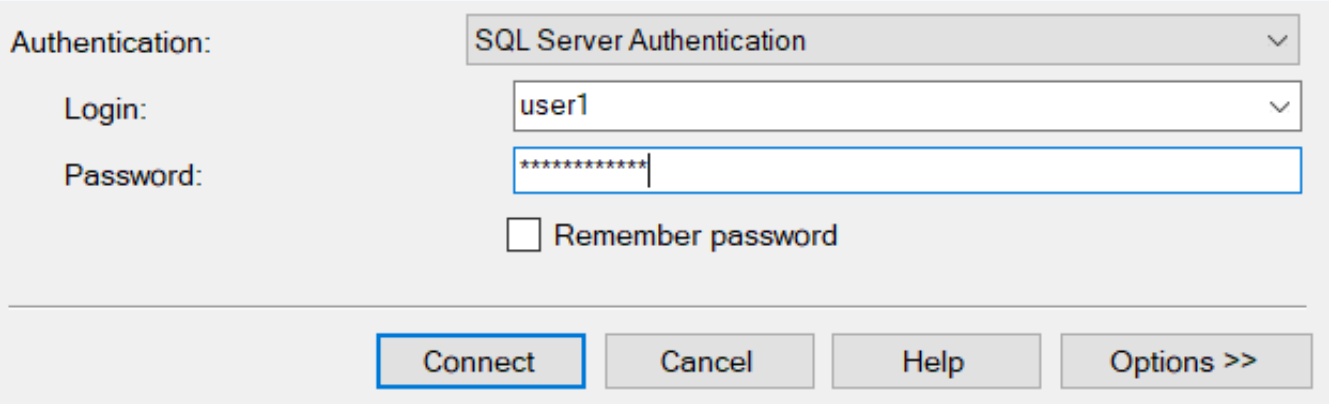
- Windows-level principals:
  - Windows local login
  - Windows domain login
  - Windows group

- Server-level principals:
  - SQL Server login
  - Server role

- Database-level principals:
  - Database user
  - Database role
  - Database user mapped to SQL Server login
  - Database user mapped to a Windows login

SQL Server Login

- Also called **user login** provides a user to **connect** to the SQL Server instance.
- Can be used if and only if the server is using the mixed authentication mode.
- The user should provide both the **login name** and **password** to connect to SQL Server.

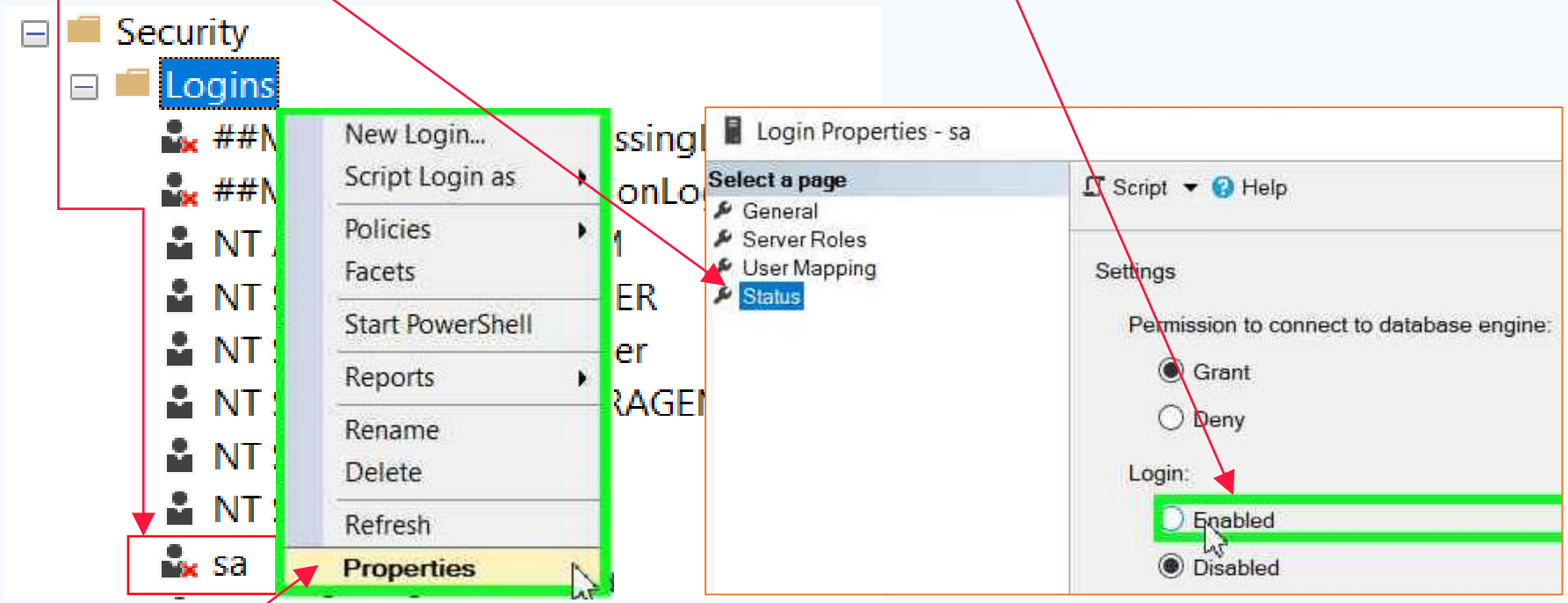


SQL Server sa Login

- The SQL Server **sa** login is a built-in server-level principal.
- SQL Server **sa** login is a member of **sysadmin** server role.
  - So it has full privileges to everything in the SQL Server environment.
- By default, it is disabled when an instance is installed, so you should enable it to login to SQL Server using it.

SQL Server sa Login…

- To enable sa login:
  - Right click on sa → Select **Properties** → **Status** page → Select ⦿**Enabled**



Creating SQL Server Login

- SQL Server login is created by DBA using the following simplified syntax:

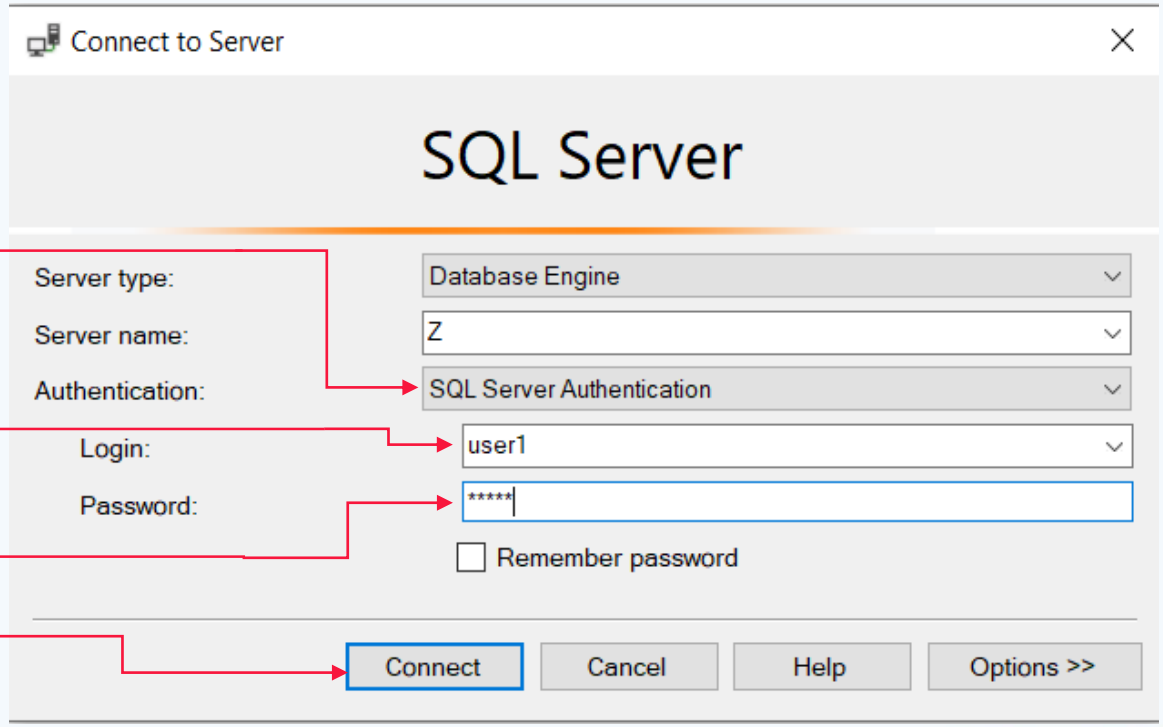
```
CREATE LOGIN loginName
WITH PASSWORD = 'password‘
```

Example: Create a login named 'user1' with a simple password 'Hi123'.

```
CREATE LOGIN user1
WITH PASSWORD = 'Hi123';
```

Connect to Server via SQL Login

- To connect (login) to a server, after creating SQL Server login, we should change the authentication to **SQL Server Authentication** in the Connect to Server window.



- Type login name
- Type password
- Click Connect

Disable, Enable, Drop SQL Login

- To disable a login or enable a disabled login:

```
ALTER LOGIN loginName
DISABLE | ENABLE;
```

Example 1: disable the login named user1.

```
ALTER LOGIN user1 DISABLE;
```

Example 2: enable the login named user1.

```
ALTER LOGIN user1 ENABLE;
```

- The syntax to delete SQL Server login:

```
DROP LOGIN loginName;
```

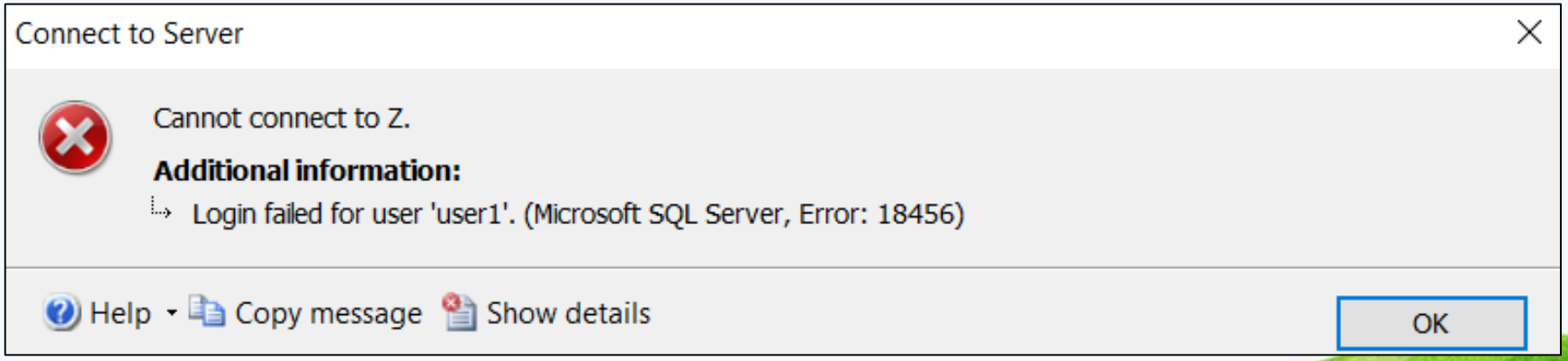
Deny Connect SQL

- To prevent the login from connecting to the SQL Server instance, use the following syntax:

  DENY CONNECT SQL LoginName;

Example: DENY CONNECT SQL user1;

- Now if you try to connect to the server with user1, you will get a connection error as follow:



Locking SQL Server Login

- Account protection mechanism in Windows causes an account to be locked out when the correct login name or password to connect to SQL Server instance is not provided within a specified number of attempts.
- If you configure the Account Lockout Policy properly in your Windows OS, SQL Server will automatically lock the login account after a few (two, three, four, …, N) wrong attempts.

Locking SQL Server Login…

- To lockout SQL Server login account, after a few wrong attempts, follow the following two steps:

1) First configure the Account Lockout Policy setting in the local security policy (**secpol.msc**) of the OS.
   - Do you know how?
2) Then enforce the password policy of the SQL login account with the option "CHECK_POLICY = ON" in SQL Server (if the account is already created and its password policy is OFF).

```
ALTER LOGIN LoginName
WITH CHECK_POLICY = ON;
```

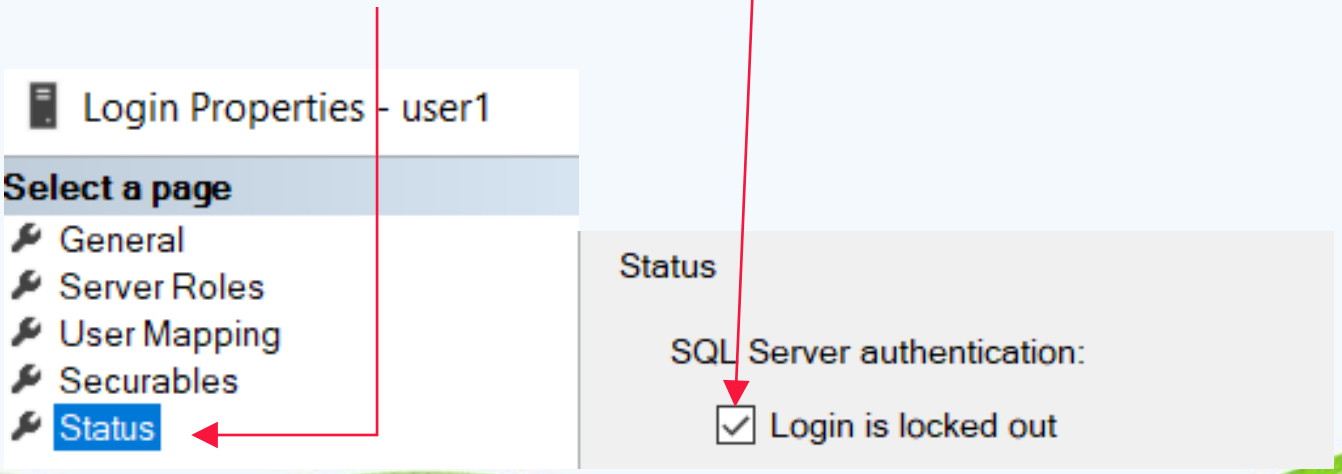
Unlocking SQL Server Login

- To unlock a locked SQL Server login account:

  ALTER LOGIN LoginName

  WITH PASSWORD = 'Password' **UNLOCK**;

- To unlock using SSMS (GUI):
  - Right click on the LoginName -> select Properties
  - Select Status -> Uncheck Login is locked out



Database Users

- Logins do not provide access to the objects contained within the database.
- Permissions to access database objects are at the database user level.
- After you have created a login to SQL Server, you need to create a database user that is mapped to the login.
- Database users are created and assigned privileges in order to access resources within a database.

Creating Database Users

- The syntax to create a database user for SQL Server login is:

```
USE DatabaseName
GO
CREATE USER UserName
FOR LOGIN LoginName;
```

- The syntax to delete a database user is:

```
DROP USER UserName;
```

Creating Database Users…

Example: Create a database user named 'hiUser' for the login 'user1' within EmployeeDB database.

```
USE EmployeeDB;
GO
CREATE USER hiUser
FOR LOGIN user1;
```

- To delete the above database user:

```
USE EmployeeDB;
DROP USER hiUser;
```

Roles

- Roles in SQL Server are principals that provide a convenient way to group multiple users with the same permissions.
- Permissions are assigned to the role, instead of individual users.
- Users then gain the required set of permissions by having their account added to the appropriate role.
- You can either create roles or use the system roles pre-defined by the DBMS.
- SQL Server provides server-level, database-level, and application roles.

Server-level Roles

- Server-level roles have a server wide permission scope.
- Also known as instance-level roles.
- SQL Server logins, Windows accounts, and Windows groups can be added to server-level roles as members.
- Server-level roles can be either fixed server roles or user-defined server roles.
- SQL Server provides nine fixed (built-in) server roles.

Fixed Server-level Roles

Role	Members can
sysadmin	Perform any action within the instance. By default, the built-in '**sa**' login and the local administrator are members of this role.
bulkadmin	Run BULK INSERT operations. But database users must be granted INSERT permission to run the Bulk Insert operations.
dbcreator	Create, alter, drop and restore databases.

Fixed Server-level Roles…

Role	Members can
securityadmin	Create, alter, and drop logins. They will be able to grant, deny, and revoke database-level and server-level permissions.
processadmin	Terminate processes running in an instance of the Database Engine.
setupadmin	Add and remove linked servers by using T-SQL. The sysadmin membership is needed when using SSMS to add linked servers.

Fixed Server-level Roles

Role	Members can
diskadmin	Manage disk resources or files (mdf, ldf).
serveradmin	Perform the same actions as diskadmin and processadmin, plus manage endpoints, change server-wide (instance) configuration settings, and shut down the instance.

- **public role:** All SQL Server logins are members of this role by default. There are no server-level permissions inherent in this role, however some server permissions are present by default, such as: CONNECT SQL.

Creating User-defined Server-level Roles

- In SQL Server 2012 and later, you can create user-defined server roles and assign server-level permissions to the user-defined server roles.
- Syntax to create user-defined server role:

```
USE master
GO
CREATE SERVER ROLE RoleName;
```

- To delete the role, use the following syntax:

```
DROP SERVER ROLE RoleName
```

Creating User-defined Server-level Roles

Example: Create Server-level role named "ManageLogins".

```
USE master
GO
CREATE SERVER ROLE ManageLogins;
```

- To delete the above role:

```
DROP SERVER ROLE ManageLogins;
```

Add Members to Server Role

- The syntax to add SQL Server login or Windows login to a server role (fixed or user-defined server role) using alter statement is:

  ALTER SERVER ROLE RoleName

  ADD MEMBER LoginName;

- You can also add the members using a system stored procedure as follow:

  EXEC SP_ADDSRVROLEMEMBER

  LoginName, RoleName;

Add Members to Server Role…

Example 1: Add SQL Server login named "user1" created so far as a member of a fixed server-level role called securityadmin.

```
ALTER SERVER ROLE securityadmin
ADD MEMBER user1;

-- Or using system stored procedure:

EXEC SP_ADDSRVROLEMEMBER
                    user1, securityadmin;
```

Add Members to Server Role…

Example 2: Add SQL Server login named "user1" created so far as a member of a user-defined server-level role called ManageLogins.

```
ALTER SERVER ROLE ManageLogins
ADD MEMBER user1;

--Or using system stored procedure:

EXEC SP_ADDSRVROLEMEMBER
                    user1, ManageLogins;
```

Remove Members of Server Role

- To remove members of a server role (fixed or user-defined server role) using alter statement:

  ALTER SERVER ROLE RoleName

  DROP MEMBER LoginName;

- To remove the member using the system stored procedure:

  EXEC SP_DROPSRVROLEMEMBER

  LoginName, RoleName;

Remove Members of Server Role…

Example 1: remove the SQL Server login named "user1" from a fixed server-level role called securityadmin.

```
ALTER SERVER ROLE securityadmin
DROP MEMBER user1;

--Or using system stored procedure:

EXEC SP_DROPSRVROLEMEMBER
                    user1, securityadmin;
```

Remove Members of Server Role…

Example 2: remove the SQL Server login named "user1" from a user-defined server-level role called ManageLogins.

```
ALTER SERVER ROLE ManageLogins
DROP MEMBER user1;

--Or using system stored procedure:

EXEC SP_DROPSRVROLEMEMBER
                    user1, ManageLogins;
```

Database-level Roles

- A database role is a principal within the database that contains one or more database users.
- Database-wide in their permissions scope.
- Types of database-level roles in SQL Server:
  1. Fixed (predefined) database roles
  2. User defined database roles
- It is recommended to create database roles, add users to a role, and then grant permissions to the role.
- It is also possible to add users to a fixed database role and inherit a fixed set of permissions.

Fixed Database Roles

Role	Members can
db_owner	Perform any action over a database and all objects within a database. Has full control over a database.
db_datareader	Select data from all tables and views within a database.
db_datawriter	Insert, update, and delete data into and from all tables within a database. To update and delete specific data, members of this role must also be members of the db_datareader role.

Fixed Database Roles (Cont'd…)

Role	Members can
db_denydatareader	Not select against all tables and views within a database.
db_denydatawriter	Not insert, update, and delete against all tables within a database.
db_ddladmin	Execute database level DDL statements (create, alter and drop table, view, procedure, …).
db_accessadmin	Add or remove users in a database.

Fixed Database Roles (Cont'd…)

Role	Members can
db_backupoperator	Back up a database but cannot restore a database or view any information in a database.
db_securityadmin	Manage the membership of database roles and associated permissions, but cannot manage membership of db_owner role.

- **public role:** Default group in every database that all database users belong to. You cannot add or remove users from it.

Creating User-defined Database Roles

- Syntax to create database role in SQL Server:

```
USE DatabaseName
CREATE ROLE RoleName;
```

Example: Create a database role named "UsersRole" within EmployeeDB database.

```
USE EmployeeDB;
CREATE ROLE UsersRole;
```

- To delete the database role, use the following syntax:

```
DROP ROLE RoleName
```

Add Members to Database Roles

- The syntax to add database users as a member of a database role (fixed or user-defined) is:

```
USE DatabaseName;
ALTER ROLE RoleName
ADD MEMBER UserName;
```

- You can also add users using a system stored procedure as:

```
USE DatabaseName;
EXEC SP_ADDROLEMEMBER RoleName,
                      UserName;
```

Add Members to Database Roles…

Example 1: Add a database user named "hiUser" created so far in EmployeeDB as a member of a fixed database role called db_datareader .

```
USE EmployeeDB;
ALTER ROLE db_datareader
ADD MEMBER hiUser;
-- Or using a system stored procedure:
USE EmployeeDB;
EXEC SP_ADDROLEMEMBER db_datareader,
                      hiUser;
```

Add Members to Database Roles…

Example 2: Add a database user named "hiUser" created so far as a member of a use-defined database role called UsersRole:

```
USE EmployeeDB;
ALTER ROLE UsersRole
ADD MEMBER hiUser;
-- Or using a system stored procedure:
USE EmployeeDB;
EXEC SP_ADDROLEMEMBER UsersRole, hiUser;
```

Remove Database Role Members

- The syntax to remove database users from a database role (fixed or user-defined) is:

```
USE DatabaseName;
ALTER ROLE RoleName
DROP MEMBER UserName;
```

- You can also remove users using a stored procedure as:

```
USE DatabaseName;
EXEC SP_DROPROLEMEMBER RoleName,
                       UserName;
```

Remove Database Role Members…

Example 1: remove a database user named "hiUser" from a fixed database role called db_datareader.

```
USE EmployeeDB;
ALTER ROLE db_datareader
DROP MEMBER hiUser;
-- Or using a system stored procedure:
USE EmployeeDB;
EXEC SP_DROPROLEMEMBER db_datareader,
                       hiUser;
```

Remove Database Role Members…

Example 2: remove a database user named "hiUser" from a user defined database role called UsersRole.

```
USE EmployeeDB;
ALTER ROLE UsersRole
DROP MEMBER hiUser;
-- Or using a system stored procedure:
USE EmployeeDB;
EXEC SP_DROPROLEMEMBER UsersRole,
                       hiUser;
```

Managing Permissions

Securables

- Securables are objects on which you grant permissions. That means, a securable is basically **"something you can secure"**.
- Every object within SQL Server, including the entire instance, is a securable.
- Securables can also be nested inside other securables.

For example, an instance contains databases; databases contain schemas; and schemas contain tables, views, procedures, and so on.

Securables Scopes

Server-scope Securables:	Database-scope Securables:	Schema-scope Securables:
▪ Database ▪ SQL Server Login ▪ Server role ▪ …	✓Database User ✓Database Role ✓Asymmetric Key ✓Symmetric Key ✓…	• Table • View • Trigger • Procedure • …

Permissions

- **Permissions** or **privileges** provide access rights to perform actions within an instance or database.
- The Database Administrator or owner's of the database object can provide or remove privileges on a database.

Permissions…

- You can assign permissions on a securable at any scope:
  - individual objects
  - schema scope
  - database scope
- You can grant or deny all permissions directly and separately on the lowest-level objects (such as tables and views).

Permissions…

- If a user needs the same permissions on all objects within a schema, you can grant or deny all permissions on the schema instead.
  - causes the permissions to be granted or denied implicitly on all objects within a schema.
- If a user needs the same permissions on all objects within a database, you can grant or deny all permissions on the database instead.
  - causes the permissions to be granted or denied implicitly on all schemas within the database and thereby on all objects within all schemas.

Permissions…

- The basic types of permissions include:
  - **DML (SELECT, INSERT, UPDATE, DELETE):** for database objects.
  - **DDL (CREATE, ALTER, DROP):** for database objects and the database itself.
  - **EXECUTE:** for stored procedures
  - **CONNECT SQL**: for SQL Server instance.
  - …

Authorization Specification in SQL

- The following DCLs are used to enforce database security in a multiple user database environment.
  - **GRANT**: to assign or give privileges.
  - **REVOKE**: to remove already assigned privileges.
  - **DENY**: to prevent privileges (this the default for non DBA users).

Grant DML Permissions

- The syntax of GRANT command on individual objects:

  GRANT PrivilegeList **ON** Object_name

  TO {user_name **|** role_name}**;**

  - PrivilegeList can be SELECT, UPDATE, DELETE, …
  - Object_name can be table name, view name, …
  - user_name can be any database user name.
  - role_name can be any database role name.

Grant DML Permissions…

- To grant the same permissions on all objects within a schema (schema scope):

  GRANT PrivilegeList **ON SCHEMA::**SchemaName

  TO {user_name **|** role_name}**;**

- To grant the same permissions on all objects within a particular database (database scope):

  GRANT PrivilegeList **ON DATABASE::**DBName

  TO {user_name **|** role_name}**;**

Remove DML Permissions

- Syntax for REVOKE command on individual objects:

  REVOKE PrivilegeList **ON** object_name

  FROM {user_name **|** role_name}**;**

- To remove the same permissions on all objects within a schema (schema scope):

  REVOKE PrivilegeList

  ON SCHEMA:: SchemaName

  FROM {user_name **|** role_name}**;**

Remove DML Permissions…

- To remove the same permissions on all objects within a particular database (database scope):

```
REVOKE PrivilegeList
ON DATABASE::DBName
FROM {user_name | role_name};
```

Example: GRANT and REVOKE

Example: give a privilege to hiUser so that the user can select and update data from employee table.

```
GRANT SELECT, UPDATE ON Employee
TO hiUser;
```

- To remove the select and update privileges given to hiUser on employee table:

```
REVOKE SELECT, UPDATE ON Employee
FROM hiUser;
```

Deny DML Permissions

- All non DBAs are denied most DML permissions by default. So no need to deny these permissions.
- However, if you want to explicitly deny these permissions, you can do so using DENY command.
- The syntax for DENY command is the same as GRANT, simply replace GRANT with DENY.

Role Based Permissions

- Steps of database-level role-based access control:
  1. Create login accounts for different users.
  2. Create database users for each login accounts.
  3. Create a database role under your database.
  4. Then grant or deny database scope privilege(s) for the role.
  5. Finally add all database users to the role as its member.
- If you want to add users to fixed database roles and inherit their permissions, skip steps 3 and 4.

Role Based Permissions…

Example: Assume we have already created SQL login named "user1", database user named "hiUser" for user1 login and a database role named "UsersRole" within EmployeeDB database. Grant the select and update privileges to this role on employee table and add hiUser as a member of this role.

```
GRANT SELECT, UPDATE ON Employee
TO UsersRole;

EXEC SP_ADDROLEMEMBER UsersRole, hiUser;
```

- Now hiUser can select data from employee table and update employee table.

DML Permissions Using Views

- Authorization mechanism using **views** can be accomplished by:
  - **Vertical view:** to give or prevent privileges on specific columns of a table. Notice that insert privilege may or may not be implemented by vertical view. Why?
  - **Horizontal view:** to give or prevent privileges on only certain rows of a table.
- Users can be given access privilege to view without allowing access privilege to the underlying tables.
  - Granting a privilege on a view does not imply granting any privileges on the underlying tables.

DML Permissions on Specific Columns

- DML permissions on sub-entity lists (specific columns).
- Can be implemented using a vertical view.
- Alternatively you can grant the privileges on specific columns (sub-entity lists) of a table directly as follow:

  GRANT PrivilegeLists **ON** TableName(ColName)

  TO UserName**;**

  - PrivilegeLists should be **UPDATE** or **SELECT**.

- Note: **INSERT** and **DELETE** privileges cannot be granted on specific columns of a table. Why?

Cont'd…

Example: suppose that the database administrator wants to allow a database user named "hiUser" to select and update only the name of Employee within the EmployeeDB database; the administrator can assign the privilege as follow:

```
GRANT SELECT, UPDATE
ON Employee(Fname, Lname)
TO hiUser;
```

- **Q?** Try to implement the above privileges using a view.

Exercises

1. What is loginless database user? How do you create loginless database user in SQL Server using T-SQL?
2. What is impersonation? How do you impersonate the loginless database user?
3. List and explain some of the common Server scope privileges in SQL Server?
4. Show the steps of server-level role-based access control or permission in SQL Server?
5. How do you grant, deny and revoke DDL permissions (CREATE, ALTER and DROP) in SQL Server using T-SQL?

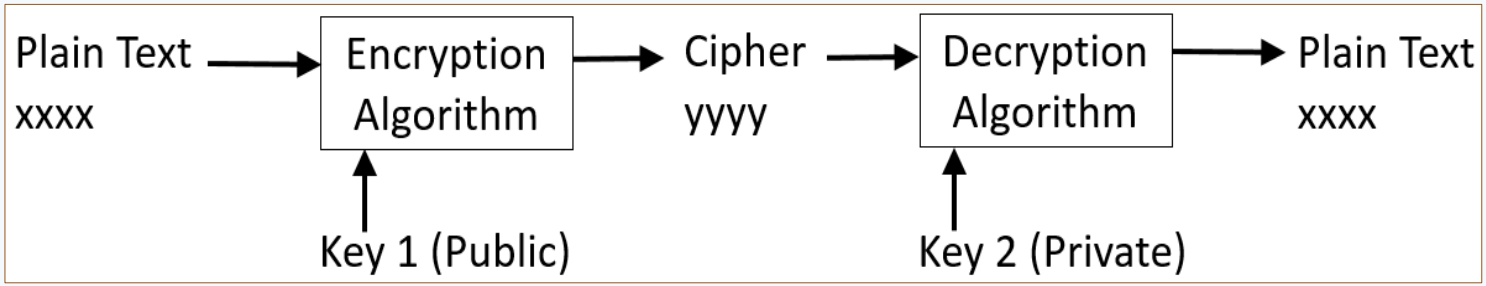
Data Encryption

Concept of Encryption

- **Encryption** is the process of encoding information in such a way that it is unreadable by unauthorized users.
- Uses different **encryption algorithms** to encode data using an **encryption key**.
- A **key** is a value supplied to an algorithm (a mathematical function) to encrypt or decrypt data.

Concept of Encryption…

- The encrypted data has to be **decrypted** using a **decryption key** to recover the original data.

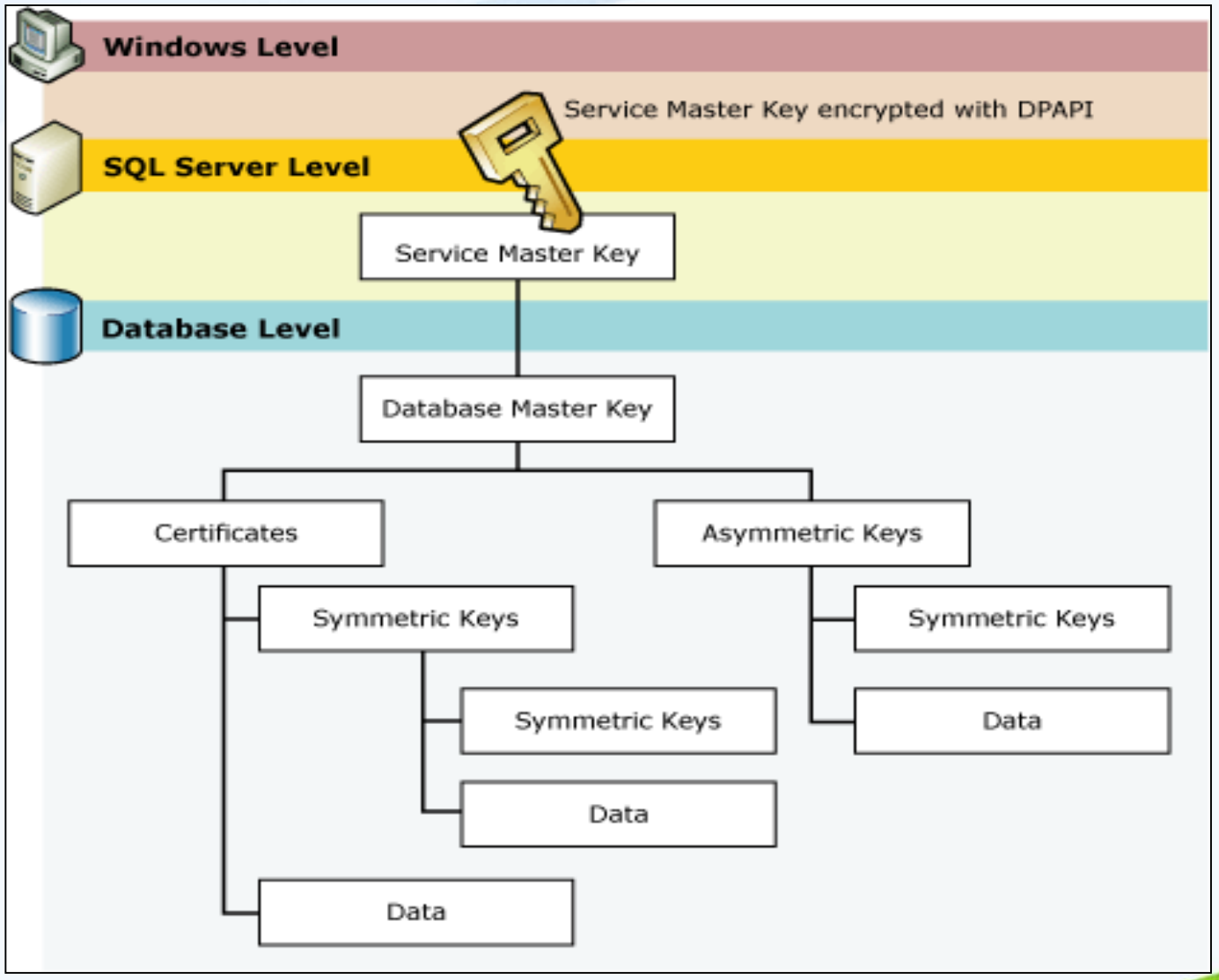


- Common encryption algorithms in SQL Server:
  - ✓ Data Encryption Standard (DES) →Triple_DES
  - ✓ Advanced Encryption Standard (AES) → AES 128, AES_192, AES_256
  - ✓ RSA_4096, RSA_3072, RSA_2048, RSA_1024, RSA_512

Concept of Encryption…

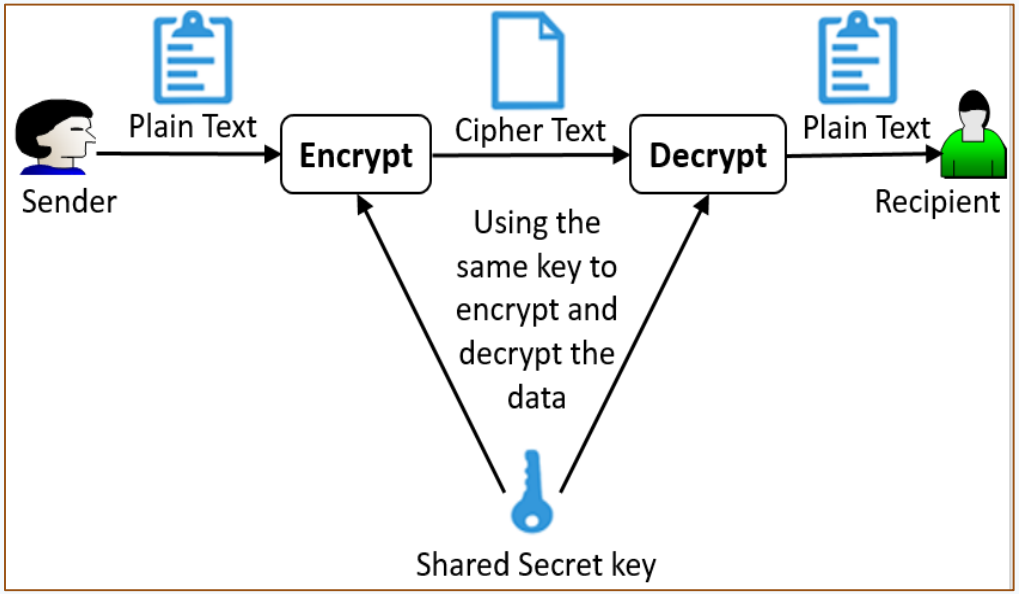
- Encrypted column must be stored as **varbinary** and then re-cast back to the appropriate data type when read.
- Common encryption techniques:
  - Certificate
  - Symmetric key Encryption Algorithm
  - Asymmetric key Encryption Algorithm
  - Hash algorithm
  - Transparent data encryption (TDE)

Key Protection Hierarchy



Symmetric Key Encryption

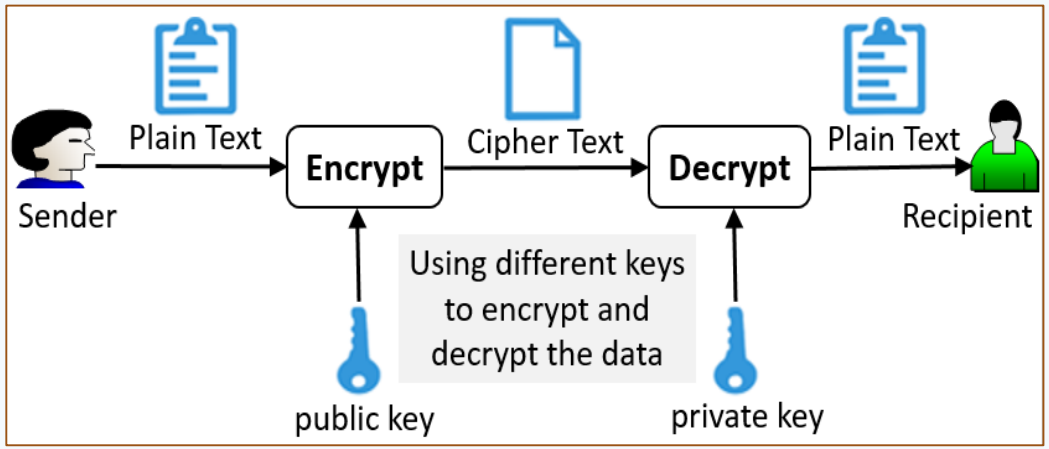
- Uses the same cryptographic keys or a single key for both encryption and decryption.
- Symmetric key encryption is not as strong as asymmetric key encryption.



- Commonly used since it provides the best possible performance for routine use of encrypted data.

Asymmetric Key Encryption

- Asymmetric key encryption uses two keys: **public** key for encryption and **private** key for decryption.



- So asymmetric keys provide the strongest encryption and decryption method like certificates.
- But slower than symmetric key encryption algorithm.

Hash Algorithms

- A **hash algorithm** is a one-way algorithm that allows you to encrypt data but does not allow decryption.
- Common hash algorithms available in SQL Server:
  - SHA, SHA1, SHA2
  - MD2, MD4, MD5
- Hash algorithms are vulnerable to brute force attacks, so confidential data should not be encrypted using a hash algorithm.

Reading Assignment

1) Data encryption using Certificates
2) Data encryption using Asymmetric key
3) Data encryption using Hash algorithm

Stored Procedures and Triggers

Stored Procedures

- When applications interact with SQL Server there are two ways to execute T-SQL code:
  - Every statement can be issued by the application
  - Groups of statements can be stored on the server as stored procedures and given a name
- Functions that are stored and executed by the DBMS are called **stored procedures**.
- Types of stored procedures in SQL Server:
  1. System Defined Stored Procedures
  2. User Defined Stored Procedures

Stored Procedures…

- Syntax to create user defined stored procedure is:

```
CREATE PROCEDURE | PROC ProcedureName
AS
 BEGIN
     <SQL code>
 END
```

- Syntax to execute user-defined stored procedure:

```
EXEC ProcedureName;
```

- Procedure may accept one or more input parameters which have @ prefix.

Stored Procedures…

Example 1: Create a procedure named "Emp_Info1" to retrieve all employees

```
CREATE PROC Emp_Info1
AS
BEGIN
        SELECT *
        FROM Employee
END
```

- Then you can execute a stored procedure as follow.

```
EXEC Emp_Info1;
```

Stored Procedures…

Example 2: Creating a stored procedure with input parameters.

```
CREATE PROC Emp_Info2(@Did CHAR(4),
                      @sex CHAR = 'F')
AS
BEGIN
   SELECT *
   FROM Employee
   WHERE Did = @Did AND Gender = @sex;
END
```

Stored Procedures…

- Now execute the stored procedure with different input parameter values:

```
EXEC Emp_Info2 'R01', 'F';

EXEC Emp_Info2 'A01', 'M';

EXEC Emp_Info2 'H01', 'M';

EXEC Emp_Info2 'A01';
```

Triggers in SQL Server

- A trigger is a special kind of stored procedure that is used to specify automatic actions that the database system will perform when certain events occur and certain conditions are satisfied.
- Events may be create, insert, select, update, delete, alter, drop, …
- Triggers are fired automatically when the events have been occurred.

Triggers in SQL Server…

- Triggers can be used in various applications, such as maintaining database consistency and monitoring database updates.
- The common types of triggers are:
  - ✓DML trigger
  - ✓DDL trigger
  - ✓Logon trigger

DML Triggers

- The simplified syntax for creating after DML trigger:

```
CREATE TRIGGER TriggerName
ON TableName | ViewName
AFTER <UPDATE, INSERT, DELETE>
AS
  PRINT 'Any notification message'
```

- To drop this type of trigger, use the syntax:

```
DROP TRIGGER TriggerName;
```

- To disable this type of trigger, use the syntax:

```
DISABLE TRIGGER TriggerName ON TableName;
```

DML Triggers …

- The syntax for creating before DML trigger:

```
CREATE TRIGGER TriggerName
ON TableName | ViewName
INSTEAD OF <UPDATE, INSERT, DELETE>
AS
  RAISERROR('Any error message', 16, 1)
```

- To drop this type of trigger, use the syntax:

```
DROP TRIGGER TriggerName;
```

- To disable this type of trigger, use the syntax:

```
DISABLE TRIGGER TriggerName ON TableName;
```

DML Triggers …

Example: Create a DML trigger named "DeptTrigger" that prevents and prints an error message to the client when anyone tries to add or change data in the Department table.

```
CREATE TRIGGER DeptTrigger
ON Department
INSTEAD OF INSERT, UPDATE, DELETE
AS
RAISERROR('You cannot modify this table',
                                    16, 1);
```

DML Triggers …

- After creating the trigger, if we try to modify data (eg. Update) in a Department table as follow:

```
UPDATE Department
SET Dname = 'Admin‘
WHERE Did = 'A01';
```

- We will get the following error message.

Msg 50000, Level 16, State 1, Procedure DeptTrigger, Line 5 [Batch Start Line 16]

You cannot modify this table

DDL Triggers

- DDL triggers are fired and provide notification in response to the execution of DDL events.
- The basic DDL events on a table and database are: CREATE_TABLE, ALTER_TABLE, DROP_TABLE, CREATE_DATABASE, ALTER_DATABASE, …
- If the DDL event executes within the context of a transaction, you can use a DDL trigger to prevent certain changes to your database schema.
- You can create either server scoped or database scoped DDL triggers.

Database-scoped DDL Triggers

- The simplified syntax to create a database scoped DDL trigger to prevent certain events from occurring:

```
USE DatabaseName;
CREATE TRIGGER TriggerName ON DATABASE
FOR | AFTER DatabaseScopedDDLEventLists
AS
BEGIN
    PRINT 'Any custom message';
    ROLLBACK;
END
```

Database-scoped DDL Triggers…

- The simplified syntax to create a database scoped DDL trigger to provide notification in response to a change:

```
USE DatabaseName;
CREATE TRIGGER TriggerName ON DATABASE
FOR | AFTER DatabaseScopedDDLEventLists
AS
BEGIN
    PRINT 'Any custom notification message';
END
```

- To modify the existing trigger, use **ALTER TRIGGER**.

Database-scoped DDL Triggers…

- To disable or enable database scoped triggers:

  DISABLE | **ENABLE TRIGGER**

  TriggerName | **ALL**

  ON DATABASE;

- To drop database scoped triggers, use the syntax:

  DROP TRIGGER TriggerName

  ON DATABASE;

Database-scoped DDL Triggers…

Example: Create a database scoped DDL trigger named "EmpTrig" to prevent alter and drop table actions within EmployeeDB database.

```
USE EmployeeDB;
CREATE TRIGGER EmpTrig ON DATABASE
FOR ALTER_TABLE, DROP_TABLE
AS
BEGIN
    PRINT 'You are prevented to …'
    ROLLBACK;
END
```

Server-scoped DDL Triggers

- The simplified syntax to create a server scoped DDL trigger to prevent certain events from occurring:

```
CREATE TRIGGER TriggerName
ON ALL SERVER
FOR | AFTER ServerScopedDDLEventLists
AS
BEGIN
    PRINT 'Any custom message';
   ROLLBACK;
END
```

Server-scoped DDL Triggers…

- The simplified syntax to create a server scoped DDL trigger to provide notification in response to a change:

```
CREATE TRIGGER TriggerName
ON ALL SERVER
FOR | AFTER ServerScopedDDLEventLists
AS
BEGIN
   PRINT 'Any custom notification message';
END
```

- To modify the existing trigger, use **ALTER TRIGGER**.

Server-scoped DDL Triggers…

- To disable or enable server scoped trigger:

```
DISABLE | ENABLE TRIGGER
    TriggerName | ALL
ON ALL SERVER;
```

- To drop server scoped trigger, use the syntax:

```
DROP TRIGGER TriggerName
ON ALL SERVER;
```

Server-scoped DDL Triggers…

Example: Create a server scoped trigger named "**EmpDBTrig**" to prevent create, alter and drop table actions within any user database and create, alter and drop database actions within the SQL Server instance:

Server-scoped DDL Triggers…

Example (Cont'd…):

```
CREATE TRIGGER EmpDBTrig ON ALL SERVER
FOR CREATE_TABLE, ALTER_TABLE,
        DROP_TABLE, CREATE_DATABASE,
        ALTER_DATABASE, DROP_DATABASE
AS
BEGIN
   PRINT 'You are not allowed to … ';
   ROLLBACK;
END
```

Reading Assignment

1. Metadata security in SQL Server
2. Ownership chains, signatures and impersonation in SQL Server
3. Auditing SQL Server Instances (Server Audit Specification vs Database Audit Specification)
4. Endpoints in SQL Server
5. Database mirroring in SQL Server